

CSC490 Assignment A5 - Testing Your Project

Yibing Ju (1009382829), Cynthia Zhou (1009028918), Jonathan Chen (1008849310), Mark Noge (1008992299)

Table of Contents:

1 Part One - Profiling Execution time	1
2 Part Two - Showing Code Coverage	19
3 Part Three: Test Coverage Breadth	21
4 Part Four: Test Coverage Depth	24
5 Part Five: Two Memorable Bugs.....	37

1 Part One - Profiling Execution Time

Since our codebase is in TypeScript and not in Python, we used a TypeScript & Node.js equivalent to Python's cProfile, **perf_hooks**, which is a Performance API

Methodology

We used Node.js's built-in `perf_hooks` module (`performance.now()`) to instrument 5 critical Lambda functions in the Peachy backend. This is the TypeScript/Node.js equivalent of Python's `profile/cProfile` module.

Approach:

1. Created a `Profiler` class (see `__tests__/profiling/profiler.ts`) that wraps `performance.now()` to collect per-operation timing data
2. For each function, ran hundreds to thousands of iterations to get statistically meaningful measurements
3. External I/O (DynamoDB, Bedrock, Expo Push API) was mocked via `aws-sdk-client-mock` to isolate CPU-bound work
4. Output follows cProfile's table format: `ncalls`, `tottime`, `percall`, `% of total`

Why did we mock I/O: In production, DynamoDB calls take 5-50ms, and Bedrock takes 500-3000ms. These dominate wall-clock time but are not optimizable in our code. By mocking them (near-zero latency), we reveal the CPU-bound bottlenecks that *we can actually improve*.

Function 1: AI Parse Handler (`lambda/ai/parse/index.ts`)

Motivation

The AI parse handler is the **most complex function in the entire codebase** (~560 lines, 15+ external calls, two execution paths). It is the entry point for Peachy's core feature -- natural-language scheduling -- and is called on every user input. Any CPU overhead here directly impacts the user-perceived latency on top of the already-expensive Bedrock AI call (500-3000ms). Profiling it reveals whether our orchestration logic adds meaningful overhead beyond the unavoidable I/O.

Before Fix: Profiling Results (200 iterations each)

Scenario	Total (ms)	Per-call (ms)
Fast path (simple input)	443.39	2.21
Smart path (no mentions)	3260.42	16.30
Smart path (@mention)	531.28	2.65
Fast path (empty calendar)	383.16	1.91

Analysis

The **smart path with no mentions** is 7.4x slower than the fast path (16.3ms vs 2.2ms per call). The input "plan a team lunch sometime next week" triggers the `parseDateHint` "next week" branch, which returns a 7-day date range. `fetchEventsForUser` then generates 7 days x N calendars parallel DynamoDB queries. The `calculateFreeWindows` function scans 168 hourly slots against all busy blocks. Additionally, `validateEventTiming` called `Intl.DateTimeFormat` constructors repeatedly (see Function 3).

Proposed Fix

No direct code change to this handler. The improvements come indirectly from **Fix 1** (`DateTimeFormat` cache in `date-utils.ts`) and **Fix 3** (Thompson sampling `topK`), both of which are called by this handler on every invocation.

Commit

Improved indirectly by commits `a16d832` (Fix 1+2) and `0ae1668` (Fix 3) on branch `dev`.

After Fix: Profiling Results (200 iterations each)

Scenario	Per-call Before (ms)	Per-call After (ms)	Improvement
Fast path (simple input)	2.21	1.98	10% faster
Smart path (no mentions)	16.30	3.94	76% faster
Smart path (@mention)	2.65	3.55	~same (noise)
Fast path (empty calendar)	1.91	2.51	~same (noise)

The **smart path** saw a dramatic 76% improvement, dropping from 16.3ms to 3.9ms per call. This is primarily due to the `DateFormat` cache eliminating repeated formatter construction across the 7-day date range processing and the `validateEventTiming` pipeline

Function 2: Update Event Handler ([lambda/events/update-event/index.ts](#))

Motivation

The update-event handler has the **most complex state management** in the codebase. DynamoDB's sort key includes `startTime`, so changing an event's time requires a delete-and-recreate pattern rather than a simple update. This handler also manages invitee notifications (pending items + push notifications). We profiled it to measure the CPU cost of its update expression building (11 repeated if-blocks) and the delete-recreate path.

Before Fix: Profiling Results (500 iterations each)

Scenario	Total (ms)	Per-call (ms)
In-place update (title only)	608.85	1.22

Delete-recreate (startTime change)	796.90	1.59
Multi-field + new invitees	1610.25	3.22

Analysis

The **update expression builder** used 11 nearly-identical if-blocks (lines 174-247), each performing 3 Map insertions with string allocations. The multi-field + invitees path was 2.6x slower than simple updates due to notification logic overhead.

Proposed Fix (Fix 4): Data-driven field mapping loop

Replaced the 11 repetitive if-blocks with a single `UPDATABLE_FIELDS` array and a loop:

```
const UPDATABLE_FIELDS = [
  { bodyKey: 'title', dbKey: 'Title' },
  { bodyKey: 'description', dbKey: 'Description' },
  { bodyKey: 'endTime', dbKey: 'EndTime', transform: (v: string) => new Date(v).toISOString() },
  // ... 8 more fields
];

for (const { bodyKey, dbKey, transform } of UPDATABLE_FIELDS) {
  if (body[bodyKey] !== undefined) {
    updateExpressions.push(`#${bodyKey} = :${bodyKey}`);
    names[`#${bodyKey}`] = dbKey;
    values[`:${bodyKey}`] = transform ? transform(body[bodyKey]) : body[bodyKey];
  }
}
```

This reduces ~70 lines to ~15, eliminates redundant string allocations, and supports a `transform` function for fields like `endTime` that need conversion.

Commit

4785d8a on branch dev

After Fix: Profiling Results (500 iterations each)

Scenario	Per-call Before (ms)	Per-call After (ms)	Improvement
In-place update (title only)	1.22	0.56	54% faster
Delete-recreate (startTime change)	1.59	0.74	53% faster
Multi-field + new invitees	3.22	2.59	20% faster

Function 3: Date Utilities ([lambda/ai/parse/date-utils.ts](#))

Motivation

`parseDateHint` and `validateEventTiming` are called on **every AI parse request** -- they are the pre-processing and post-processing stages that bracket the Bedrock call. `parseDateHint` scopes the DynamoDB date-range query (directly affecting how many database calls are made), and `validateEventTiming` runs a 7-step correction pipeline on the AI's output. These are pure functions with no I/O, so any CPU bottleneck here is entirely within our control. Initial profiling revealed they were the **single largest CPU bottleneck** in the codebase due to `Intl.DateTimeFormat` construction.

Before Fix: Profiling Results

`parseDateHint` (5000 iterations each)

Input Type	Per-call (ms)
------------	---------------

<code>bareDayName</code> ("soccer on wednesday")	0.258
<code>nextWeek</code> ("review session next week")	0.191
<code>prefixedDay</code> ("this friday lunch")	0.175
<code>nextDay</code> ("next monday standup")	0.173
<code>tomorrow</code>	0.095
<code>noHint</code> (default to today)	0.089
<code>inDays</code> ("in 3 days")	0.089
<code>inWeeks</code> ("in 2 weeks")	0.089
<code>todayDefault</code>	0.088

parseDateHint Timezone Impact (5000 iterations, same input)

Timezone	Per-call (ms)	Relative to no-tz
No timezone	0.004	1.0x (baseline)
America/Toronto	0.266	66.5x slower
Asia/Tokyo	0.261	65.3x slower

UTC	0.295	73.8x slower
Europe/London	0.297	74.3x slower

validateEventTiming (5000 iterations each)

Scenario	Per-call (ms)
explicit-time-fix (step 6)	1.078
unreasonable-hour (step 3)	0.984
late-night-explicit	0.924
valid-no-adjust	0.808
wrong-day-fix (step 5)	0.802
end-before-start (step 4)	0.796
past-date-fix (step 1)	0.791
conflict-snap (step 7, 6 windows)	0.791
conflict-snap (42 windows)	3.776

Analysis

The dominant bottleneck is `Intl.DateTimeFormat` construction. Timezone-aware calls are **66-74x slower** than non-timezone calls. Every call to `getLocalHour()`, `getLocalDayOfWeek()`, and

`getLocalDateParts()` created a new `Intl.DateTimeFormat` instance. `validateEventTiming` calls these helpers ~7 times across its 7 steps.

The **conflict check with 42 free windows** (3.78ms) was 4.8x slower than with 6 windows because it called `new Date()` and `getLocalHour()` for every window inside the loop.

Proposed Fix (Fix 1 + Fix 2)

Fix 1: Cache `Intl.DateTimeFormat` instances. Added a module-level `Map<string, Intl.DateTimeFormat>` cache and a `getCachedFormatter()` helper. Updated all three timezone helpers to use it:

```
const formatterCache = new Map<string, Intl.DateTimeFormat>();

function getCachedFormatter(timezone: string, options: Intl.DateTimeFormatOptions): Intl.DateTimeFormat {
  const key = `${timezone}:${JSON.stringify(options)}`;

  let fmt = formatterCache.get(key);

  if (!fmt) {
    fmt = new Intl.DateTimeFormat('en-US', { ...options, timeZone: timezone });
    formatterCache.set(key, fmt);
  }

  return fmt;
}
```

Fix 2: Pre-parse free window timestamps. Refactored the conflict check loop (step 7) to pre-parse all window ISO strings to epoch milliseconds and pre-compute local hours *once* before the loop, instead of calling `new Date()` and `getLocalHour()` inside the loop for each window.

Commit

a16d832 on branch dev

After Fix: Profiling Results

parseDateHint (5000 iterations each)

Input Type	Before (ms)	After (ms)	Improvement
------------	-------------	------------	-------------

bareDayName	0.258	0.010	96% faster
nextWeek	0.191	0.008	96% faster
prefixedDay	0.175	0.008	96% faster
nextDay	0.173	0.009	95% faster
tomorrow	0.095	0.009	91% faster
noHint	0.089	0.006	93% faster
inDays	0.089	0.006	93% faster
inWeeks	0.089	0.006	93% faster
todayDefault	0.088	0.006	93% faster

parseDateHint Timezone Impact (5000 iterations, same input)

Timezone	Before (ms)	After (ms)	Relative to no-tz (after)
No timezone	0.004	0.002	1.0x
America/Toronto	0.266	0.009	5.6x (was 66.5x)
Asia/Tokyo	0.261	0.010	6.3x (was 65.3x)

UTC	0.295	0.009	5.6x (was 73.8x)
Europe/London	0.297	0.009	5.6x (was 74.3x)

Timezone overhead reduced from 66-74x to 5-6x -- a ~12x improvement in the gap.

validateEventTiming (5000 iterations each)

Scenario	Before (ms)	After (ms)	Improvement
explicit-time-fix	1.078	0.222	79% faster
unreasonable-hour	0.984	0.232	76% faster
late-night-explicit	0.924	0.059	94% faster
valid-no-adjust	0.808	0.044	95% faster
wrong-day-fix	0.802	0.044	95% faster
end-before-start	0.796	0.060	92% faster
past-date-fix	0.791	0.046	94% faster
conflict-snap (6 windows)	0.791	0.045	94% faster
conflict-snap (42 windows)	3.776	0.230	94% faster

Function 4: User Search Handler (`lambda/users/search/index.ts`)

Motivation

User search is called whenever someone types in the "Add Friend" or "Invite to Event" flows. It uses a **3-tier search fallback** (GSI exact match -> GSI prefix -> DynamoDB Scan), which is an interesting pattern to profile because the cost profile changes dramatically depending on which tier handles the request. We wanted to verify whether the scan fallback was causing unnecessary work and measure the overhead of batch friendship status checks.

Profiling Results (500 iterations each)

Before (no code change)

Scenario	Per-call (ms)
Username search	0.79
Email search	0.61
Scan-heavy (50 results)	0.44

After (no code change -- included for completeness)

Scenario	Per-call (ms)
Username search	0.43
Email search	0.37
Scan-heavy (50 results)	0.38

Analysis

CPU-bound work is minimal (0.4-0.8ms). The scan fallback is already conditional (only runs when GSI returns fewer than `limit` results). No code change was applied. The slight improvement in "after" numbers is noise from the test environment, not a real change.

No fix applied. The function is already efficient for its CPU-bound work. The real production bottleneck would be the DynamoDB Scan I/O cost, which is an architectural concern (acceptable for MVP with small user base).

Commit

No fix applied -- no commit for this function.

Function 5: Thompson Sampling ([lambda/shared/src/thompson.ts](#))

Motivation

Thompson sampling powers Peachy's **time-slot preference learning** -- it's the core of our reinforcement learning feature. It runs on every AI parse request to rank 168 time slots (7 days x 24 hours) by learned user preference. The function uses advanced statistical algorithms (Marsaglia & Tsang Gamma sampling, Box-Muller transform, Beta distribution via Gamma ratio). We profiled it to understand whether the mathematical operations or the data structure overhead dominated, and whether we could reduce work given that callers only need the top 5 slots.

Before Fix: Profiling Results

Full thompsonSampleAll (1000 iterations)

Weights	Total (ms)	Per-call (ms)
Sparse (10% filled)	503.85	0.50
Dense (90% filled)	425.29	0.43

Sub-operation Breakdown (1000 iterations, moderate weights)

Operation	Per-call (ms)	% of total
sampleBeta x168	0.370	89.5%
buildSlots	0.022	5.4%

sort	0.020	4.8%
------	-------	------

sampleBeta Parameter Sensitivity (10,000 iterations)

Parameters	Per-call (ms)	Notes
Beta(0.5, 0.5) Jeffreys	0.0025	Slowest -- triggers Gamma shape<1 boost trick
Beta(1, 1) uniform	0.0021	Default for new slots
Beta(50, 20) strong	0.0020	Many observations
Beta(5, 3) weak	0.0020	Few observations

Analysis

Beta sampling dominates at 89.5%. Sorting all 168 slots (4.8%) is wasteful since both callers only use `.slice(0, 5)`. Sparse weights allocated a new `{ alpha: 1, beta: 1 }` default object for every missing slot (up to 168 per call).

Proposed Fix (Fix 3): Pre-allocated default + topK parameter

1. **Pre-allocated default prior:** Added `const DEFAULT_PRIOR: BetaParams = { alpha: 1, beta: 1 }` at module level, eliminating up to 168 small object allocations per call.
2. **Optional `topK` parameter:** When provided, uses a selection approach (maintain sorted array of K best) instead of full $O(n \log n)$ sort. Both callers updated to pass `topK: 5`.

```
export function thompsonSampleAll(weights: Record<string, BetaParams>, topK?: number): RankedSlot[] {
```

```
  // ... sampling loop uses DEFAULT_PRIOR instead of { alpha: 1, beta: 1 } ...
```

```
  if (topK !== undefined && topK < slots.length) {
```

```
    //  $O(n * K)$  selection beats  $O(n \log n)$  sort when  $K \ll n$ 
```

```
    const best: RankedSlot[] = [];
```

```
    for (const slot of slots) {
```

```

    if (best.length < topK) {

      best.push(slot);

      best.sort((a, b) => b.score - a.score);

    } else if (slot.score > best[best.length - 1].score) {

      best[best.length - 1] = slot;

      best.sort((a, b) => b.score - a.score);

    }

  }

  return best;

}

// ... full sort fallback ...

}

```

Commit

0ae1668 on branch dev

After Fix: Profiling Results

Full thompsonSampleAll (1000 iterations) -- no topK, tests the DEFAULT_PRIOR improvement

Weights	Before (ms)	After (ms)	Improvement
Sparse (10% filled)	0.50	0.67	~same (noise)
Dense (90% filled)	0.43	0.55	~same (noise)

Sub-operation Breakdown (unchanged -- sampleBeta still dominates)

Operation	Before (ms)	After (ms)	% of total
-----------	-------------	------------	------------

sampleBeta x168	0.370	0.550	90.3%
buildSlots	0.022	0.033	5.4%
sort	0.020	0.024	3.9%

The per-call numbers vary between runs due to the stochastic nature of the sampling algorithm (different random seeds, different numbers of rejection sampling iterations). The structural improvements (DEFAULT_PRIOR, topK selection) are confirmed correct by passing all 211 unit tests. The topK optimization eliminates the full sort in production use (callers now pass topK: 5), though the profiling sub-operation test still exercises the full-sort path.

Summary: Before vs After

Function	Metric	Before	After	Change
parseDateHint (bareDayName, with tz)	per-call	0.258ms	0.010ms	96% faster
parseDateHint tz overhead	ratio vs no-tz	66-74x	5-6x	~12x better
validateEventTiming (avg scenario)	per-call	0.893ms	0.095ms	89% faster
validateEventTiming (42 windows)	per-call	3.776ms	0.230ms	94% faster
updateEvent (in-place)	per-call	1.22ms	0.56ms	54% faster

updateEvent (delete-recreate)	per-call	1.59ms	0.74ms	53% faster
aiParse (smart, no mentions)	per-call	16.30ms	3.94ms	76% faster
thompsonSampleAll	structural	full sort 168	topK=5 selection	Correct, noise in timing
userSearch	per-call	0.79ms	0.43ms	No change (noise)

How to Run the Profiling Scripts

cd Infra/Peachy-Infra

Run all profiling tests

npx jest __tests__/profiling/ --no-coverage --verbose

Run individual function profiles

npx jest __tests__/profiling/profile-thompson.test.ts --no-coverage

npx jest __tests__/profiling/profile-date-utils.test.ts --no-coverage

npx jest __tests__/profiling/profile-user-search.test.ts --no-coverage

npx jest __tests__/profiling/profile-update-event.test.ts --no-coverage

npx jest __tests__/profiling/profile-ai-parse.test.ts --no-coverage

Files

Profiling Infrastructure

File	Purpose
__tests__/profiling/profiler.ts	Shared profiling utility (wraps perf_hooks)

<code>__tests__/profiling/profile-thompson.test.ts</code>	Profiles <code>thompsonSampleAll</code> , <code>sampleBeta</code> , <code>sampleGamma</code>
<code>__tests__/profiling/profile-date-utils.test.ts</code>	Profiles <code>parseDateHint</code> , <code>validateEventTiming</code>
<code>__tests__/profiling/profile-user-search.test.ts</code>	Profiles user search handler
<code>__tests__/profiling/profile-update-event.test.ts</code>	Profiles update-event handler
<code>__tests__/profiling/profile-ai-parse.test.ts</code>	Profiles AI parse handler (fast + smart paths)

Source Files Modified

File	Fix	Change
<code>lambda/ai/parse/date-utils.ts</code>	Fix 1 + 2	Added <code>Intl.DateTimeFormat</code> cache; pre-parsed conflict loop timestamps
<code>lambda/shared/src/thompson.ts</code>	Fix 3	Added <code>DEFAULT_PRIOR</code> constant; added <code>topK</code> parameter with selection
<code>lambda/ai/parse/index.ts</code>	Fix 3	Updated <code>thompsonSampleAll</code> call sites to pass <code>topK: 5</code>
<code>lambda/events/update-event/index.ts</code>	Fix 4	Replaced 11 if-blocks with <code>UPDATABLE_FIELDS</code> data-driven loop

Git References

- **Branch:** `dev`
- **Commit (profiling scripts):** `618dadb`

- **Commit (Fix 1+2 -- date-utils cache + conflict pre-parse):** a16d832
- **Commit (Fix 3 -- Thompson topK + default prior):** 0ae1668
- **Commit (Fix 4 -- update-event field loop):** 4785d8a

2 Part Two - Showing Code Coverage

Peachy is split across two repositories: **Peachy** (React Native frontend in <https://github.com/UofT-CSC490-W2026/Peachy/tree/dev>) and **Peachy-Infra** (AWS Lambda backend in <https://github.com/UofT-CSC490-W2026/Peachy-Infra/tree/dev>). Both use **Jest** with the `--coverage` flag, which is the standard JavaScript/TypeScript equivalent of `pytest-cov`.

Running Coverage Locally

Both repos use the same command:

```
npm run test:coverage
```

The backend requires the shared Lambda layer to be compiled first:

```
npm run build:layer && npm run test:coverage
```

Both produce a full coverage report in the terminal and write reports to a `coverage/` directory in `text`, `lcov`, and `json-summary` formats.

GitHub Actions Workflows

Both repositories have a CI workflow at `.github/workflows/test.yml` that runs automatically on every push and pull request to `main` or `dev`.

Full workflow files:

- **Frontend:**
<https://github.com/UofT-CSC490-W2026/Peachy/blob/dev/.github/workflows/test.yml>
- **Backend:**
<https://github.com/UofT-CSC490-W2026/Peachy-Infra/blob/dev/.github/workflows/test.yml>

Coverage Badges

Both READMEs display four live badges (lines, statements, functions, branches) that update automatically on every push to `main` or `dev`:

- **Frontend:** github.com/UofT-CSC490-W2026/Peachy/tree/dev
- **Backend:** github.com/UofT-CSC490-W2026/Peachy-Infra/tree/dev

Badges are powered by `schneegans/dynamic-badges-action` writing to a GitHub Gist, read by shields.io.

On every **pull request**, `ArtiomTr/jest-coverage-report-action` automatically posts a coverage summary table as a PR comment showing per-file coverage and any regressions.

After every CI run, the full HTML coverage report is also available as a downloadable artifact:

GitHub Actions → **Select a run** → **Artifacts** → **coverage-report**

- **Frontend:** <https://github.com/UofT-CSC490-W2026/Peachy/actions>
- **Backend:** <https://github.com/UofT-CSC490-W2026/Peachy-Infra/actions>

The artifact contains:

- `lcov.info` — machine-readable (compatible with Codecov, SonarQube, etc.)
- `coverage-summary.json` — JSON summary used for badges and PR comments
- `index.html` — browsable HTML showing line-by-line coverage per file

3 Part Three: Test Coverage Breadth

Coverage Summary

Repository	Statements	Branches	Functions	Lines
Peachy (frontend)	98.47%	93.6%	95.94%	99.44%
Peachy-Infra (backend)	92.7%	84.52%	91.72%	93.47%

Full per-file breakdown available as a downloadable HTML report from each CI run:

- <https://github.com/UofT-CSC490-W2026/Peachy/actions>
 - <https://github.com/UofT-CSC490-W2026/Peachy-Infra/actions>
-

What Is Covered

Frontend:

- All utility functions, form components, calendar components, custom hooks, and constants are fully unit tested
- `ai-input-bar.tsx` is tested including the async AI send flow, recording start/stop, transcription, error handling, and permission denial
- Components are tested using `@testing-library/react-native` with `renderWithProviders` wrapping all theme-dependent UI

Backend:

- Lambda handlers tested across all domains: calendars, events, chats, friends, users, pending items, RL, AI parsing, and transcription
- Each handler is tested for validation errors, auth failures, not found cases, permission checks, and happy paths
- CDK infrastructure stacks verified with `aws-cdk-lib/assertions` (synthesizes stacks and checks CloudFormation output without deploying)
- Shared Lambda layer utilities (auth, DynamoDB client, error classes, Thompson Sampling, validators) fully unit tested

- Fire-and-forget flows (push notifications, pending item creation) are covered via integration with mocked AWS SDK clients
- **AI integration tests** (`__tests__/integration/ai/parse.test.ts`, `ai/parse-mentions.test.ts`) test the real AWS Bedrock API end-to-end — these are run manually against the deployed dev environment and excluded from CI since they require AWS credentials and a live endpoint, as mentioned in part 4.

What Is Not Testes

The following are excluded from the Jest test suite because they cannot be meaningfully exercised without live infrastructure or native device APIs:

Area	Reason
<code>utils/notifications.ts</code>	Native Expo push notification APIs unavailable in Jest
<code>utils/api-client.ts</code>	HTTP client — requires integration tests against real API
<code>utils/ai-parser.ts</code>	Mock stub for AWS Bedrock (no real logic to test)
<code>components/ui/**</code>	SF Symbols native module not available in Jest
<code>components/haptic-tab.tsx</code>	Native haptic feedback
Calendar grid views (<code>month-view</code> , <code>week-view</code> , <code>day-view</code> , etc.)	Pixel-positioned visual components with no testable logic

<code>components/availability-viewer.tsx</code>	Requires live CalendarContext + API
Live AWS Transcribe / Bedrock calls	Require deployed infrastructure and credentials
Real Expo push delivery	Requires live Expo push service
DynamoDB retry backoff logic	Timeout-based retry that cannot be reliably mocked

4 Part Four: Test Coverage Depth

Module Under Test

lambda/ai/parse/ is Peachy's AI scheduling pipeline. This Lambda takes natural language input (e.g., "soccer on thursday", "watching movie for 3 hours on saturday"), calls AWS Bedrock (Claude) to extract event details, validates and corrects the AI's output, and returns structured event fields (title, startTime, endTime, isAllDay, location) ready for calendar creation.

Primary test files:

- **tests/ai/parse.test.ts** (20 handler-level integration tests for single-user scheduling)
- **tests/ai/parse-mentions.test.ts** (10 handler-level integration tests for multi-person friend scheduling)

Why We Chose This Module

The AI scheduling pipeline is the most complex feature in Peachy. A user types a sentence, and the system must:

1. Parse the date hint: extract "thursday", "tomorrow", "saturday" from natural language to determine which day's calendar to fetch
2. Fetch existing events: query DynamoDB (GSI3-DateRange) for the user's calendar events on that day
3. Compute free windows: subtract busy blocks from the day to find available time slots
4. Call the AI: send context to AWS Bedrock (Claude) with the free windows and user preferences
5. Validate the output: run 7 sequential checks to catch AI errors (wrong day, wrong time, timezone confusion, double-booking) before writing to the database

For multi-person scheduling, the pipeline adds additional steps: 6. Resolve @mentions: extract @brian, @sarah from input, look up usernames via GSI1 7. Filter by friendship: only friends can be added as event participants 8. Compute mutual availability: fetch each participant's calendar and find windows where everyone is free

Every bug in this pipeline results in events at the wrong time, on the wrong day, or double-booked, which is directly visible to users. Three real bugs discovered during testing motivated these tests (see Part 5).

Test Design

All 30 tests call the real Lambda handler function with realistic natural language inputs. DynamoDB and Bedrock are mocked (using aws-sdk-client-mock) so no real AWS calls are made, but all internal logic runs: date parsing, event fetching, free window calculation, AI response parsing, @mention resolution, friendship filtering, mutual availability computation, and the full 7-layer validation pipeline.

Each test sends a natural language string through handler() and asserts on the event fields that come back, which are the same fields that would be used to create a calendar event in production.

Describe Block	Tests	Type
Day only, no time	3	Positive, realistic inputs
Time only, no day	2	Positive, time-only input + evening UTC conversion
Day + time	3	Positive, field population + location
Timezone-aware Validation	3	Regression (Part 5 bugs) + negative
AI Output Correction	3	Regression + negative
Conflict Avoidance	3	Regression (part 5 bug) + positive
Error handling	3	Negative, bad input + service failure
Friend scheduling, day + friend	3	Positive, mention resolution + friend inclusion
Friend conflict avoidance	2	Regression + positive, mutual availability
Multi-friend + non-friend + edge cases	5	Positive + negative, multi-participant scheduling

Detailed Test Descriptions

Event creation, day only, no time (Tests 1-3)

These test what happens when a user specifies a day but no specific time, so the AI must pick a reasonable time.

#	Input	What It Tests	Why This Test?
1	"soccer on thursday"	Bare day name → event on correct Thursday, title populated, isAllDay false	Positive test. Most common input pattern. Verifies parseDateHint picks the next Thursday, Bedrock's title/time pass through, and all event fields are populated.
2	"watching movie for 3 hours on saturday"	Duration preservation, Bedrock returns 3-hour event (7pm-10pm EDT)	Positive test. Users specify durations naturally. Asserts endTime - startTime = exactly 3 hours. Verifies the handler doesn't truncate or modify the duration.
3	"vacation on saturday"	All-day event, isAllDay = true	Positive test. No time or duration specified. Bedrock recognizes this as an all-day event. Asserts isAllDay is true and title is populated. Tests the only non-timed event type in the handler.

Event creation, time only, no day (Tests 4-5)

Users often say just a time without specifying which day. `parseDateHint` defaults to today.

#	Input	What It Tests	Why This Test?
4	"soccer at 7pm"	Time-only input, Bedrock infers the day	Positive test. Verifies the handler accepts time-only input and the 7pm local time (23:00 UTC for EDT) passes through correctly.
5	"dinner at 6pm"	Evening time, correct EDT-to-UTC conversion	Positive test. 6pm in Toronto is 22:00 UTC. Verifies the handler correctly converts evening local time to UTC and passes it through without modification.

Event creation, day + time (Tests 6-8)

The most explicit input pattern: user specifies both day and time.

#	Input	What It Tests	Why This Test?
6	"tennis tomorrow at 3pm"	Full specification → exact <code>startTime</code> and <code>endTime</code>	Positive baseline. The "golden path" where the user says exactly what they want, Bedrock returns it correctly,

			and the handler passes it through. Asserts exact ISO timestamps.
7	"birthday party saturday at noon"	"noon" keyword handling	Positive test. "noon" is treated as an explicit time (12pm). Verifies noon EDT = 16:00 UTC and routes through fast path correctly.
8	"team meeting tomorrow at 10am" + Bedrock returns location	Location field passthrough	Positive test. Bedrock can return an optional location field. Asserts it appears in the response alongside correct title and times.

Timezone-aware validation (Tests 9-11)

These test the timezone bugs discovered in Part 5. The Lambda runs in UTC but users are in EDT (UTC-4).

#	Input	What It Tests	Why This Test?
9	"meeting tomorrow" + Bedrock returns 06:00 UTC (= 2am EDT)	Unreasonable hour caught in local timezone	Regression test (Bug 1). 6am UTC looks fine, but it's 2am in Toronto. Verifies the handler adjusts to 9am local (13:00 UTC) and adds an adjustment warning to ambiguities.

10	"meeting tomorrow" + Bedrock returns 06:00 UTC (= 2am EDT)	Explicit time enforcement catches UTC/local confusion	Regression test (Bug 2). Bedrock treated "4pm" as UTC instead of converting. 16:00 UTC = 12pm local, not 4pm. Verifies correction to 20:00 UTC (4pm local).
11	"gaming at 2am tomorrow" + Bedrock returns 06:00 UTC (= 2am EDT)	Explicit late-night keyword prevents over-correction	Critical negative test. The user intentionally said "2am". The unreasonable-hour check would normally adjust this, but "2am" in the input triggers the EXPLICIT_LATE_NIGHT exemption. Verifies the handler respects the user's intent and does NOT adjust.

AI output correction (Tests 12-14)

These test the post-AI validation layer that catches and corrects Bedrock errors.

#	Input	What It Tests	Why This Test?
12	"soccer on wednesday" + Bedrock returns Tuesday	Day-name enforcement corrects wrong day	Regression test (Bug 2). Bedrock returned Tuesday March 31 for a Wednesday request. Verifies the handler shifts to Wednesday April 1.

13	"meeting wednesday" + Bedrock returns past Wednesday	Past-date check advances to next occurrence	Regression test. Bedrock returned March 25 (past Wednesday). Verifies the handler advances by exactly 7 days to April 1 (next Wednesday).
14	"midnight movie tomorrow" + Bedrock returns midnight (04:00 UTC)	"midnight" keyword exempts from unreasonable-hour check	Negative test. Midnight (hour 0) would normally be flagged as unreasonable (range: 11pm-5am). The "midnight" keyword in the input exempts it. Verifies no adjustment occurs.

Conflict avoidance (Tests 15-17)

These test that the handler prevents double-booking by checking AI output against the user's existing calendar events.

#	Input	What It Tests	Why This Test?
15	"quick sync tomorrow" + user busy 10am-12pm, Bedrock picks 11am	Event during busy block → snapped to free window	Regression test (Bug 3). Bedrock ignored the user's existing events and picked a conflicting time. Verifies the handler moves the event outside the busy block.
16	"coffee tomorrow" + user free all day,	No conflicts → passes	Positive baseline. When there are no conflicts, the

	Bedrock picks 2pm	through unchanged	handler must not modify the time. Asserts exact startTime/endTime match Bedrock's output.
17	"lunch tomorrow" + two busy blocks (9-11am, 1-3pm), Bedrock picks noon gap	Event in gap between busy blocks → no conflict	Positive test. Bedrock correctly picked the gap. Verifies the handler doesn't move an event that's already in a valid free window, even with multiple surrounding busy blocks.

Error handling (Tests 18-20)

Negative tests verifying the handler gracefully rejects bad input and handles service failures.

#	Input	What It Tests	Why This Test?
18	Missing inputText	Request body validation	Negative test. Handler must return 400 before calling Bedrock. Guards against empty requests.
19	Invalid timezone "Fake/Zone"	Timezone validation	Negative test. Invalid IANA timezone strings must be rejected before any date math occurs.
20	Valid input but Bedrock	External service failure	Negative test. When Bedrock is unavailable, the handler

	throws	handling	returns 500 with an error message rather than crashing.
--	--------	----------	---

Multi-Person Friend Scheduling (Tests 21-30)

Test file: **tests/ai/parse-mentions.test.ts**, 10 handler-level tests in the "realistic multi-person scheduling" describe block

These tests exercise the full @mention → friend resolution → mutual availability → conflict avoidance pipeline. Each test sends natural language with @friend mentions through the real Lambda handler and verifies that friends are resolved, non-friends are excluded, and events avoid everyone's busy blocks.

Friend scheduling, day + friend (Tests 21-23)

#	Input	What It Tests	Why This Test?
21	"play soccer with @brian on thursday at 4pm"	Friend resolved with explicit time, correct day	Positive test. @brian resolves via GSI1-Username, passes friendship check, and appears in invitedUserIds. Tests that @mentions force smart path even with an explicit time (bug fix, see Design Decisions).
22	"watching movie for 3 hours with @brian on saturday"	3-hour duration preserved with friend	Positive test. Same duration preservation as Test 2, but with a friend participant. Asserts endTime - startTime = exactly 3 hours AND Brian is in invitedUserIds.

23	"dinner with @brian tomorrow at 8pm"	Evening time near UTC day boundary with friend	Positive test. 8pm EDT = midnight UTC the next day. Tests that friend resolution works correctly even when the event time crosses the UTC day boundary.
----	--------------------------------------	--	---

Friend conflict avoidance (Tests 24-25)

#	Input	What It Tests	Why This Test?
24	"study with @brian tomorrow" + Brian busy 2-4pm	Friend's busy block avoided	Regression test. Bedrock picks 3pm (inside Brian's busy block). Verifies the handler checks the friend's calendar and snaps the event to a free window that works for both people.
25	"lunch with @brian tomorrow" + caller busy 10am-12pm + Brian busy 1-3pm	Both calendars checked, avoids both busy blocks	Positive test. Two different busy blocks on two different calendars. Verifies the handler computes mutual availability (not just the caller's free windows) and places the event in a gap that works for everyone.

Multi-friend scheduling (Tests 26-27)

#	Input	What It Tests	Why This Test?
26	"coffee with @brian and @sarah tomorrow at 3pm"	Two friends resolved and included	Positive test. Both @brian and @sarah resolve, both pass friendship check, both appear in invitedUserIds with length 2. Tests the multi-participant pipeline with explicit time.
27	"hang out with @brian and @sarah tomorrow" + staggered busy blocks	Two friends with overlapping busy blocks → event avoids both	Edge case. Brian busy 10am-12pm, Sarah busy 11am-1pm, their blocks overlap but aren't identical. Verifies the handler computes the union of busy blocks and finds a slot outside both.

Non-friend exclusion and edge cases (Tests 28-30)

#	Input	What It Tests	Why This Test?
28	"play basketball with @stranger tomorrow at 5pm"	Non-friend excluded from invitedUserIds	Negative test. @stranger resolves to a real user but is NOT friends with the caller. Verifies they're excluded from invitedUserIds and reported in unresolvedMentions.

29	"movie with @brian tomorrow" + Brian busy all afternoon	Friend busy block forces time shift	Positive test. Brian is busy 12-6pm EDT and Bedrock picks 2pm (inside busy). Verifies the handler shifts the event outside the busy block entirely.
30	"study with @brian tomorrow at 2pm" + both free	No conflicts → passes through unchanged	Positive baseline. When neither the caller nor the friend has any events, the handler must not modify the time. Asserts exact startTime/endTime match Bedrock's output.

Running the Tests

cd Peachy-Infra npx jest **tests/ai/parse.test.ts tests/ai/parse-mentions.test.ts**

Output: Test Suites: 2 passed, 2 total Tests: 30 passed, 30 total

Additional test files

Beyond the 30 focused handler tests above, we have additional tests for the same module:

- **tests/ai/parse-date-utils.test.ts** (30 tests): Unit tests for the `parseDateHint()` and `validateEventTiming()` functions directly. These test all 7 natural-language date patterns and all 7 validation checks with injectable now and timezone parameters for full determinism.
- **tests/ai/parse-mentions-unit.test.ts** (14 tests): Handler-level tests for `@mention` extraction, username resolution, friendship filtering, and conflict-aware mutual availability scheduling with multiple participants.

npx jest **tests/ai/** Test Suites: 4 passed, 4 total Tests: 74 passed, 74 total

Design Decisions

Why mock DynamoDB with IndexName matching instead of TableName? The dynamodb.ts module evaluates `process.env.USERS_TABLE_NAME` at import time, before Jest sets test environment variables. This means TableName is undefined in all DDB calls at test runtime. We match on IndexName and ExpressionAttributeValues instead, which are set at call time and reliably available.

Why use tool_use format for Bedrock mocks? The handler uses `tool_choice: { type: 'tool', name: 'parse_event' }` which forces Claude to respond with a tool_use content block. The mock must match this format. Using a plain text block would cause the handler to throw "AI returned an unexpected response format."

Why realistic natural language inputs? Each test uses input that a real user would type ("soccer on thursday", "watching movie for 3 hours on saturday", "dinner at 6pm") rather than synthetic test strings. This ensures we're testing the full pipeline as users actually experience it, including how `analyzeInput()` routes between fast path (Haiku) and smart path (Sonnet).

Bug discovered during testing: @mentions silently ignored with explicit times. While writing the friend scheduling tests (Tests 21-30), we discovered that `analyzeInput()` checked for explicit times before checking for @mentions. Inputs like "dinner with @brian at 8pm" had exactly 1 explicit time, which triggered the fast-path early return, skipping mention resolution entirely. The friend was silently dropped from `invitedUserIds`. We fixed this by moving the @mention check above the explicit-time check, so any input containing @mentions always routes to the smart path where friend resolution occurs. Tests 21, 23, 26, 28, and 30 now verify this fix by combining @mentions with explicit times.

5 Part Five: Two Memorable Bugs

The following descriptions and reflections on the bug are also discussed in the video, but are elaborated on here in text form.

https://drive.google.com/file/d/130yU_VWiscD7zjmQ5sDT9Rfci18CNsp/view?usp=sharing

Peachy dark mode display bug

- The bug consisted of cases where black text would display on a dark text field background. It was discovered during visual testing as part of a PR review before merge.
- The cause was simple to identify: the text color values were hard coded instead of being theme aware. However, two developers independently encountered this bug when working with Claude code, which was problematic.
- The bug itself was fixed by replacing the hard coded colors with theme aware classes and function calls (we have a `ThemeContext` context as source of truth and wrappers like `<ThemedText>` and `<ThemedView>`).
- The main lesson was that although AI coding agents are powerful, there are steps we can take to improve our efficiency working with Claude and each other. For example, utilizing and especially sharing files for other features such as subagents, tools, and planning to ensure consistent output across developers.
- To prevent future occurrences of this bug, we added unit tests to make sure components actually use theme-aware classes and `useThemeColor()` in their style sheets instead of hard coding values. Furthermore, we added descriptions on how accessibility and dark mode are handled in Claude context files such as `CLAUDE.md` to prevent the agent from making the same mistake.

<https://drive.google.com/file/d/118YLwN-ZDg2ctXNhrO9rO8mgskLuX4kZ/view?usp=sharing>

Peachy ai bug

Correction: I misspoke at around 1:43; 2PM UTC is 10AM EST, not the other way around. The Lambda layer understood the input as UTC instead of EST which caused the error. Also, here EST is with daylight savings.

- This was a suite of bugs related to our AI parse functionality in three main categories: time mismatch, day mismatch, and scheduling conflicts. Essentially, the output we receive from the AI appeared to not adhere to user input, or would not take the user's availability into account when scheduling. This bug was again discovered during manual testing before merge.
- By tracing the execution of the code, we discovered the main cause for the error, which is that the user's timezone was not properly threaded through the Lambda functions: it was used in `callBedrock()` to format for the prompt, but not in `parseDateHint()` which used `new Date()` instead (which is UTC on Lambda). Similarly, the existing `validateEventTiming()` (which checks for

correctness such as end-before-start and event not in the past) used `getUTCHours()`.

Additionally, we found that there is no existing validation for Bedrock output with respect to user input; that is, we do not check that the output actually matches what the user requested.

- We fixed this by actually passing the timezone parameter into the functions that require them, as well as adding verification for AI output with respect to the user input in the form of day-name enforcement, explicit time enforcement, and conflict checks against free windows.
- We learned to be mindful of UTC on serverless (previous tests were only carried out in UTC which is why we missed this) and add tests in user time. We also learned to not trust LLM output and validate everything just in case. A future step is to A-B test different models/model architectures to discover the best choice for both efficiency and correctness.
- To prevent reoccurrences, we added positive and negative tests for each one of these fail modes and more (see Part 4 for an in depth discussion on testing the AI parse functionality).